

NovaTech Solutions Pvt. Ltd.

Software Release Process

Document ID: OPS-REL-001

Version: 2.0

Prepared By: Engineering & DevOps Team

Effective Date: January 2026

Classification: Internal Use Only

Document Control

Field	Value
Document	Software Release Process
Department	Engineering
Version	2.0
Owner	Release Management Office
Review Cycle	Every 6 Months
Last Updated	January 2026

Table of Contents

1. Introduction
 2. Purpose
 3. Scope
 4. Release Definition
 5. Release Types
 6. Roles and Responsibilities
 7. Release Lifecycle Overview
 8. Revision History
-

1. Introduction

The Software Release Process defines the standard methodology for planning, building, testing, approving, deploying, and validating software releases for the NovaDesk platform.

A structured release process minimizes deployment risk, improves software quality, and ensures predictable delivery of new features, enhancements, and bug fixes.

This document applies to:

- Frontend applications
 - Backend services
 - REST APIs
 - Database migrations
 - Background workers
 - Infrastructure-related application deployments
-

2. Purpose

The objectives of the Software Release Process are to:

- Deliver high-quality software.
- Reduce production deployment risk.
- Standardize release activities.
- Improve deployment reliability.
- Enable predictable release schedules.
- Maintain auditability.
- Ensure appropriate testing and approvals.
- Support rapid recovery when issues occur.

Every production deployment should follow this release process unless emergency release procedures apply.

3. Scope

This process governs software releases affecting:

- Web applications
- Mobile backend services
- APIs
- Scheduled jobs
- Database schema migrations
- Configuration changes packaged with releases

- Container images
- CI/CD deployment pipelines

Operational infrastructure changes without application releases are governed by the Change Management Process.

4. Release Definition

A release is a planned deployment of software changes into an environment.

A release may include:

- New features
- Bug fixes
- Performance improvements
- Security updates
- Configuration updates
- Database migrations
- Dependency updates

Each release receives a unique Release ID and version number.

Release Components

A production release may contain:

- Application binaries
- Container images
- Configuration files
- Database migration scripts
- Static assets
- API updates
- Documentation updates
- Release notes

All release artifacts should be version-controlled.

5. Release Types

NovaTech categorizes releases according to business purpose and urgency.

Major Release

Major releases introduce significant functionality.

Characteristics:

- New capabilities
- Architectural enhancements
- Large feature sets
- Significant testing effort

Typical frequency:

Quarterly

Examples:

- New workflow engine
 - Major UI redesign
 - Platform architecture upgrade
-

Minor Release

Minor releases provide incremental improvements.

Characteristics:

- Feature enhancements
- Small improvements
- Limited configuration changes

Typical frequency:

Monthly

Examples:

- Additional reporting features
 - Workflow improvements
 - User interface enhancements
-

Maintenance Release

Maintenance releases improve platform stability.

Typical contents:

- Bug fixes
- Performance optimizations
- Security patches
- Dependency updates

Typical frequency:

Biweekly

Hotfix Release

Hotfix releases address urgent production issues.

Examples:

- Critical defects
- Security vulnerabilities
- Production outages
- High-impact customer issues

Hotfixes follow an expedited approval process and require a retrospective review.

6. Roles and Responsibilities

Successful releases require collaboration across multiple teams.

Release Manager

Responsible for:

- Coordinating release activities.
- Scheduling deployments.
- Tracking release readiness.
- Managing release communications.
- Confirming deployment completion.

The Release Manager owns the overall release timeline.

Development Team

Responsible for:

- Implementing approved features.
- Fixing defects.
- Completing code reviews.
- Writing automated tests.
- Updating technical documentation.

Developers are responsible for code quality before release.

Quality Assurance (QA)

Responsible for:

- Functional testing.
- Regression testing.
- Integration testing.
- User Acceptance Testing (UAT) coordination.
- Defect verification.

QA confirms release readiness before approval.

DevOps Team

Responsible for:

- Build generation.
- CI/CD pipeline execution.
- Deployment automation.
- Infrastructure readiness.
- Rollback support.

DevOps engineers monitor deployments during implementation.

Product Owner

Responsible for:

- Prioritizing release scope.
- Approving business functionality.

- Reviewing release objectives.
- Confirming feature completeness.

The Product Owner represents business requirements.

Information Security

Responsible for reviewing releases that affect:

- Authentication
- Authorization
- Encryption
- Security controls
- Sensitive data handling

Security approval is required for releases affecting security functionality.

7. Release Lifecycle Overview

Every software release follows a standardized lifecycle.

Planning

↓

Development

↓

Code Review

↓

Testing

↓

Release Approval



Deployment



Validation



Monitoring



Release Closure

Each phase should be documented within the Release Management System.

Release Status Values

Standard release states include:

Status	Description
Planned	Release scope defined
In Development	Features under implementation
Code Complete	Development completed
Testing	QA activities in progress
Approved	Ready for deployment
Scheduled	Deployment window assigned

Status	Description
Deploying	Release in progress
Validating	Post-deployment verification
Released	Successfully deployed
Closed	Documentation completed

Status changes should be recorded throughout the release lifecycle.

Release Record

Each release record should include:

- Release ID
- Version number
- Release type
- Business objectives
- Included features
- Bug fixes
- Known issues
- Testing evidence
- Approval history
- Deployment schedule
- Rollback strategy
- Release notes
- Validation results
- Closure summary

Release records provide traceability and support operational audits.

Revision History

Version	Date	Description
1.0	January 2024	Initial Software Release Process
1.5	July 2025	Added Release Categories and Governance

Version Date	Description
2.0	January 2026 Updated Roles, Lifecycle, and Release Documentation Requirements

8. Release Planning

Release planning begins after development scope has been approved by the Product Owner. Planning ensures that all required activities are coordinated before deployment.

Planning Activities

The Release Manager coordinates:

- Feature selection
- Sprint completion review
- Risk identification
- Resource allocation
- Deployment scheduling
- Maintenance window confirmation
- Stakeholder communication
- Release documentation

Planning should begin at least two weeks before a major production release.

Release Readiness Checklist

Before deployment planning proceeds, verify:

- Development completed
- Code reviews approved
- Testing completed
- Security review completed
- Documentation updated
- Rollback plan prepared
- Monitoring dashboards ready
- Change approvals received

Only releases meeting all readiness criteria should proceed.

9. Code Freeze

A code freeze prevents unnecessary changes immediately before production deployment.

Objectives

The code freeze period allows teams to:

- Stabilize the release
- Complete testing
- Resolve critical defects
- Prepare deployment artifacts

Feature development pauses during this period.

Code Freeze Rules

During code freeze:

- No new features may be merged.
- Only approved bug fixes are permitted.
- Critical security fixes require Release Manager approval.
- Documentation updates remain permitted.

Exceptions require documented approval.

10. Build Process

After code freeze, the build process generates deployable release artifacts.

Build Activities

The build pipeline performs:

- Source code compilation
- Dependency resolution
- Static code analysis
- Unit test execution
- Container image creation

- Artifact packaging
- Digital signing
- Artifact publication

Build failures must be resolved before deployment continues.

Versioning

NovaDesk follows Semantic Versioning.

Format:

MAJOR.MINOR.PATCH

Example:

4.6.2

Where:

- **MAJOR** — Breaking or significant functionality changes
- **MINOR** — Backward-compatible feature additions
- **PATCH** — Bug fixes and maintenance updates

Version numbers are recorded in the Release Management System.

11. CI/CD Pipeline

NovaDesk uses an automated Continuous Integration and Continuous Deployment (CI/CD) pipeline.

Automation improves consistency, quality, and deployment speed.

Pipeline Stages

Standard pipeline stages include:

1. Source Code Checkout
2. Dependency Installation
3. Static Code Analysis
4. Unit Testing
5. Security Scanning
6. Build Generation
7. Container Image Creation

8. Artifact Storage
9. Deployment to Test Environment
10. Automated Validation
11. Deployment Approval
12. Production Deployment

Each stage must complete successfully before the next begins.

Pipeline Validation

The pipeline automatically verifies:

- Build success
- Test completion
- Security scan results
- Artifact integrity
- Container image availability
- Deployment readiness

Failed validation blocks production deployment.

12. Testing Requirements

Software quality is verified through multiple testing stages.

Required Testing

The following testing activities are required:

- Unit Testing
- Integration Testing
- Functional Testing
- Regression Testing
- Performance Testing
- Security Testing
- User Acceptance Testing (UAT)

Testing scope depends on release type and associated risk.

Exit Criteria

Testing is considered complete when:

- Critical defects are resolved.
- High-severity defects are addressed or formally accepted.
- Regression testing passes.
- Security findings are reviewed.
- Product Owner approves business functionality.

Testing evidence must be attached to the release record.

13. Release Approval

Production deployment requires formal approval.

Required Approvals

Depending on release scope, approvals may include:

- Release Manager
- Quality Assurance Lead
- Product Owner
- DevOps Lead
- Information Security (if applicable)
- Change Manager (for production changes)

Approvals are recorded electronically.

Release Go/No-Go Meeting

For Major Releases, a Go/No-Go meeting is conducted before deployment.

Agenda includes:

- Release readiness review
- Testing summary
- Outstanding risks
- Rollback readiness

- Infrastructure status
- Business approval

The Release Manager documents the final deployment decision.

14. Production Deployment

Deployments are executed during approved maintenance windows unless emergency release procedures apply.

Deployment Steps

Typical deployment sequence:

1. Notify stakeholders.
2. Verify infrastructure readiness.
3. Create deployment backup.
4. Deploy application artifacts.
5. Execute database migrations.
6. Restart application services.
7. Verify application health.
8. Perform smoke testing.
9. Enable customer traffic.
10. Begin post-release monitoring.

Each deployment activity is documented.

Deployment Verification

Immediately after deployment, verify:

- Application startup
- API availability
- Database connectivity
- Authentication
- Scheduled jobs
- Monitoring dashboards
- Logging

- Customer access

Deployment is not considered complete until validation succeeds.

15. Rollback Strategy

Every release must include a tested rollback procedure.

Rollback Triggers

Rollback should be considered if:

- Critical functionality fails.
- Smoke tests fail.
- Customer impact exceeds acceptable limits.
- Severe performance degradation occurs.
- Security issues are identified.

Rollback decisions should prioritize service stability.

Rollback Procedure

The rollback process includes:

1. Suspend deployment activities.
2. Restore previous application version.
3. Restore previous configuration.
4. Reverse database migration (where supported).
5. Validate restored services.
6. Notify stakeholders.

Rollback activities should be documented.

16. Release Communication

Clear communication ensures stakeholders remain informed throughout the release process.

Internal Communication

Release notifications should include:

- Release version
- Planned deployment time
- Expected impact
- Maintenance window
- Rollback strategy
- Contact information

Engineering teams should receive detailed deployment instructions.

Customer Communication

Where customer impact is expected, communication should include:

- Planned maintenance window
- Service availability expectations
- Known limitations
- Expected completion time
- Support contact information

Customer notifications should be reviewed before publication.

Release Completion Notification

After successful deployment, stakeholders receive confirmation including:

- Release version
- Deployment status
- Validation summary
- Known issues
- Next planned release (if applicable)

Completion notifications formally conclude the deployment phase.

17. Deployment Best Practices

Release teams should:

- Deploy only approved artifacts.
- Verify backups before deployment.

- Monitor deployments continuously.
- Validate services immediately after deployment.
- Maintain detailed deployment records.
- Execute rollback promptly when required.
- Communicate progress regularly.
- Document implementation deviations.
- Preserve deployment logs.
- Conduct post-release reviews for significant releases.

Consistent deployment practices improve software reliability and customer confidence.

18. Post-Release Validation

After deployment, the release team must verify that the application is operating correctly in the production environment.

Validation should be completed before the release is formally closed.

Validation Activities

The release team should verify:

- Application availability
- User authentication
- API functionality
- Database connectivity
- Background jobs
- Scheduled tasks
- Monitoring dashboards
- Log collection
- Customer-facing workflows
- Third-party integrations

Validation results must be recorded in the Release Management System.

Acceptance Criteria

A release is considered successful when:

- All deployment steps complete successfully.
- Smoke tests pass.
- Critical business functions operate normally.
- No Priority 1 (P1) or Priority 2 (P2) defects remain.
- Monitoring shows stable system performance.
- Required stakeholders approve release completion.

If validation fails, the rollback procedure should be initiated according to the approved release plan.

19. Post-Release Monitoring

After deployment, production systems enter an enhanced monitoring period.

Monitoring Activities

Engineering teams monitor:

- Application availability
- API response times
- Error rates
- CPU utilization
- Memory utilization
- Database performance
- Background job execution
- Infrastructure health

Monitoring helps identify issues that may not appear during pre-production testing.

Enhanced Monitoring Window

Standard monitoring periods:

Release Type	Enhanced Monitoring
Major Release	24 Hours
Minor Release	8 Hours
Maintenance Release	4 Hours

Release Type	Enhanced Monitoring
---------------------	----------------------------

Hotfix	12 Hours
--------	----------

The Release Manager may extend monitoring if elevated operational risk is identified.

20. Hotfix Process

A Hotfix is an expedited software release used to resolve urgent production issues.

Hotfix Criteria

A Hotfix may be approved for:

- Critical production defects
- Security vulnerabilities
- Service outages
- Data integrity issues
- High-impact customer incidents

Hotfixes should address only the immediate issue whenever possible.

Hotfix Workflow

1. Incident identification.
2. Technical assessment.
3. Emergency approval.
4. Development and testing.
5. Production deployment.
6. Validation.
7. Monitoring.
8. Post-Implementation Review.

The standard release process resumes after the emergency has been resolved.

Hotfix Documentation

Every Hotfix must include:

- Hotfix ID

- Related Incident ID
- Root cause summary
- Testing evidence
- Deployment timeline
- Validation results
- Lessons learned

Documentation should be completed promptly after implementation.

21. Release Metrics and KPIs

Release Management effectiveness is evaluated through operational metrics.

Key Performance Indicators

Metric	Description
Release Success Rate	Percentage of successful releases
Failed Release Rate	Releases requiring rollback
Deployment Frequency	Number of production deployments
Mean Deployment Time	Average deployment duration
Rollback Rate	Percentage of releases requiring rollback
Release Lead Time	Time from approval to deployment
Change Failure Rate	Releases causing production incidents
Post-Release Defects	Defects reported after deployment

Metrics are reviewed monthly by Engineering Leadership.

Management Reporting

Monthly reports include:

- Total releases
- Releases by type
- Deployment success trends
- Rollback statistics

- Hotfix frequency
- Change-induced incidents
- Process improvement recommendations

Reports support engineering planning and operational governance.

22. Documentation Requirements

Complete release documentation ensures traceability and audit readiness.

Required Records

Each release should include:

- Release ID
- Version number
- Business objectives
- Feature list
- Defect list
- Testing evidence
- Deployment plan
- Rollback plan
- Approval history
- Validation results
- Release notes
- Closure summary

Documentation must accurately reflect the deployed software.

Audit Requirements

Operational audits verify:

- Process compliance
- Approval records
- Deployment evidence
- Testing documentation

- Rollback readiness
- Release validation
- Closure documentation

Audit findings are tracked until corrective actions are completed.

23. Continuous Improvement

Release Management processes should evolve based on operational experience.

Improvement Activities

Engineering teams should regularly:

- Review failed releases.
- Improve deployment automation.
- Reduce manual deployment steps.
- Strengthen testing practices.
- Enhance monitoring coverage.
- Improve rollback procedures.
- Update release documentation.
- Conduct release retrospectives.

Continuous improvement increases release reliability over time.

Knowledge Sharing

Lessons learned should be shared through:

- Engineering meetings
- Sprint retrospectives
- Internal Knowledge Base articles
- Technical documentation
- DevOps training sessions

Knowledge sharing promotes consistent release quality across teams.

24. Compliance and Governance

Software releases must comply with organizational governance requirements.

Governance Principles

Release Management should ensure:

- Clearly defined ownership
- Standardized release procedures
- Required approvals
- Deployment traceability
- Audit readiness
- Continuous review

Governance responsibilities are shared between Engineering, DevOps, Quality Assurance, Information Security, and Product Management.

Policy Compliance

Production releases should comply with:

- Change Management Process
- Information Security Policy
- Secure Development Guidelines
- Infrastructure Standards
- Quality Assurance Procedures
- Regulatory requirements

Exceptions require documented approval.

25. Release Best Practices

Release teams should:

- Maintain predictable release schedules.
- Deploy only tested artifacts.
- Automate repetitive deployment tasks.
- Keep rollback procedures current.
- Validate services immediately after deployment.
- Monitor production systems closely.

- Minimize emergency releases through proactive planning.
- Document every production deployment.
- Conduct release retrospectives.
- Continuously improve deployment automation.

Consistent release practices improve software quality and customer satisfaction.

26. Frequently Asked Questions

Q1. What is the purpose of post-release validation?

To confirm that deployed software functions correctly and meets release acceptance criteria before the release is closed.

Q2. How long should a Major Release receive enhanced monitoring?

Major Releases should receive **24 hours** of enhanced monitoring after deployment.

Q3. When should a Hotfix be used?

Hotfixes are used for urgent production issues such as critical defects, security vulnerabilities, service outages, or data integrity problems.

Q4. What is the Release Success Rate?

The percentage of production releases completed successfully without requiring rollback or causing significant operational issues.

Q5. Which documents are required for release audits?

Release plans, approvals, testing evidence, deployment records, rollback plans, validation results, and release notes.

Q6. What is the Change Failure Rate?

The percentage of software releases that result in production incidents or require corrective action.

Q7. Who reviews Release Management metrics?

Engineering Leadership reviews operational release metrics on a monthly basis.

Q8. Why are release retrospectives conducted?

To identify lessons learned, improve future release processes, and strengthen deployment quality.

Q9. What should happen after a failed release?

Execute rollback if necessary, investigate the failure, document findings, implement corrective actions, and schedule a revised release.

Q10. When is a release officially closed?

After successful validation, completion of documentation, stakeholder approval, and confirmation that post-release monitoring objectives have been met.

Revision History

Version	Date	Description
1.0	January 2024	Initial Software Release Process
1.5	July 2025	Added Hotfix Process and CI/CD Governance
2.0	January 2026	Added Post-Release Validation, Metrics, Continuous Improvement, and Audit Requirements

Related Documents

- Change Management Process (OPS-CHG-001)
 - Incident Response Process (OPS-IR-001)
 - Infrastructure Guide (TECH-INFRA-001)
 - Developer Guide (TECH-DEV-001)
 - Database Backup Manual (TECH-DB-001)
 - Information Security Policy (COMP-SEC-003)
-